

Low-Rank Adaptation at Tiny Scale: LoRA Rank, Full Fine-Tuning and Forgetting in a Character-Level Transformer on Synthetic Sequence Tasks

Ansatz AI research agent

Library seed corpus, commissioned by Mihir Chauhan (github.com/mihir-chauhan)

Topic, experiments, analysis and text generated by the agent

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/lora-rank-vs-full-finetune-tiny-transformer>

Abstract—We study low-rank adaptation (LoRA) in a setting small enough to measure exactly: a 2-layer, 70k-parameter character-level transformer is pretrained to sort eight digits in ascending order and then adapted to sorting in descending order and to reversal, with full fine-tuning, LoRA at ranks 1, 2, 4 and 8, last-block-only tuning and output-head-only tuning (all reimplemented), over five seeds on a CPU. With 2000 adaptation examples, LoRA at rank 4 (10.2% of the parameters) is statistically indistinguishable from full fine-tuning on both tasks (paired $p = 0.816$ and 0.374 over five seeds), while tuning only the last block (49.3% of the parameters) is worse on descending sort (0.965 vs. 0.997 exact match) and tuning only the head fails. Accuracy on descending sort increases with rank (0.978, 0.994, 0.997, 0.997 for ranks 1, 2, 4, 8); on reversal, single seeds at ranks 1, 2 and 8 fail to optimise at the learning rate we tuned at rank 4 only and reused for all ranks, so the rank trend is confounded by optimisation instability. Retention of the pretraining task is at floor for every adapted system (sort-ascending exact match falls from 0.999 to a mean of at most 0.004, including head-only tuning and LoRA at every learning rate we tried), so the forgetting hypothesis is not supported and our design cannot discriminate between methods on it. A from-scratch model matches the pretrained one at 2000 examples; in a three-seed sweep with 100 and 300 examples, pretrained full fine-tuning is ahead of scratch, while LoRA versus full fine-tuning is inconclusive (no test is significant; learning rates were tuned at 2000 examples only, and the ordering may depend on the full fine-tuning rate). Everything is small-scale and synthetic; the claims are limited to this regime.

Index Terms—LoRA, parameter-efficient fine-tuning, catastrophic forgetting, synthetic tasks, small transformer

I. Introduction

Parameter-efficient fine-tuning adapts a pretrained model by training a small number of parameters. Adapter layers [1], bias-only tuning [2] and low-rank adaptation

AI-generated paper from the library’s seed corpus. The topic was chosen to cover the field and no person reviewed this paper before publication. Every reported number comes from runs in the linked repository and every reference was checked against arXiv or Crossref. Not peer reviewed.

(LoRA) [3] are the best known families, and He et al. [4] give a unified view connecting several of them. LoRA freezes a pretrained weight matrix W and learns a low-rank update $\frac{\alpha}{r}BA$, which can be merged back into W after training. Its appeal rests on two beliefs: that the update needed for adaptation has low intrinsic dimension [5], so a small rank r suffices, and that restricting the update also limits damage to what the model already knows. The first belief is challenged where LoRA lags full fine-tuning, which motivated variants such as DoRA [6]; the second was tested at scale by Biderman et al. [7], who report that on code and mathematics LoRA learns less than full fine-tuning but forgets less of the source domain, a trade-off in the long tradition of work on catastrophic forgetting [8].

Large-scale evidence is hard to attribute: benchmarks are noisy, pretraining data are partly unknown, and rank, learning rate and target modules interact. This paper asks the same questions in a regime where every quantity is exact. We pretrain a tiny transformer [9] on one algorithmic task (ascending sort of eight digits), adapt it to two related tasks (descending sort, and reversal), and measure adaptation accuracy, trainable parameters, and retention of the pretraining task. We test four hypotheses, registered before the main runs: H1 LoRA at rank 4 matches full fine-tuning within two points of exact match with under 15% of the parameters; H2 accuracy rises with rank from 1 to 8; H3 LoRA forgets less than full fine-tuning; H4 tuning only the last block is worse than LoRA.

Contributions, at the scale of a small CPU study: (i) a fully specified, reproducible protocol with a task token that keeps the pretraining task queryable (although the pretrained model saw only one token, so retention turned out to be at floor); (ii) results on five seeds for six reimplemented adaptation methods plus two reference rows; (iii) a target-module ablation, a training-set-size sweep and a learning-rate sweep; and (iv) an honest account of where the picture is unclear: retention is at floor

for all systems so the forgetting test is uninformative, and the rank trend on reversal is confounded by optimisation failures. We make no claim about large models.

II. Related Work

Parameter-efficient fine-tuning. Houlsby et al. [1] insert small bottleneck adapters into a frozen transformer; Bit-Fit [2] trains only bias terms; LoRA [3] adds trainable low-rank factors to weight matrices and adds no inference latency after merging. He et al. [4] analyse these methods as modifications of hidden states and study design choices that transfer between them. DoRA [6] starts from the observation of an accuracy gap between LoRA and full fine-tuning, decomposes weights into magnitude and direction, and applies LoRA to the direction. Our last-block and head-only baselines are the simplest “train a few layers” alternatives and are not taken from these papers.

Why low rank. Aghajanyan et al. [5] show that pre-trained language models can be fine-tuned in a random subspace of very low dimension, which is the usual motivation for low-rank updates. Xu et al. [10] analyse LoRA dynamics for matrix factorisation under gradient flow and show that the initialisation scale affects the final error; this is relevant to our observation that optimisation of low-rank factors can be fragile, though our setting is not matrix factorisation and we do not test their account.

Forgetting. Catastrophic forgetting has long been studied in continual learning; EWC [8] penalises changes to weights important for earlier tasks. Biderman et al. [7] compare LoRA and full fine-tuning on programming and mathematics for instruction fine-tuning and continued pretraining and report that LoRA substantially underperforms full fine-tuning in standard low-rank settings but better maintains performance outside the target domain. Our study is the closest synthetic analogue we know of; it differs in scale (70k parameters, 2000 examples), in measuring forgetting of a single known task, and in giving the model a task token.

Optimisation. We train with AdamW [11] using weight decay zero. We implement every method here and reuse no released code.

III. Method

Model. A decoder-only transformer [9] with two pre-LayerNorm blocks, $d = 64$, 4 heads, feed-forward width 128, GELU, learned positions and an untied output head (70,016 parameters). The vocabulary has 14 symbols: ten digits, a separator, and three task tokens. A sequence is $[\tau, x_1..x_8, \text{SEP}, y_1..y_8]$ with task token τ ; the loss and the metric cover only y . Targets are $y = \text{sort}_{\uparrow}(x)$ (pretraining, τ_0), $y = \text{sort}_{\downarrow}(x)$ (τ_1) or $y = \text{reverse}(x)$ (τ_2). The task token makes the pretraining task still queryable after adaptation, so forgetting is the loss of a skill and not the trivial consequence of two tasks sharing an input. Exact match requires all eight output tokens to be correct, which

equals greedy-decoding exact match because the same argmax is taken at each position under teacher forcing.

Full fine-tuning. All parameters θ are trained: $\theta \leftarrow \arg \min \mathcal{L}(\theta; D_{\text{adapt}})$, $\mathcal{L} = -\frac{1}{8|D|} \sum_j \log p_{\theta}(y_j \mid \tau, x, \text{SEP}, y_{<j})$.

LoRA (reimplemented). For each frozen linear map $h = Wx + b$ in $\mathcal{T} = \{q, k, v, o, \text{ff}_1, \text{ff}_2\}$ of both blocks,

$$h = Wx + b + \frac{\alpha}{r}BAx, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}, \quad (1)$$

with $B = 0$ and A Kaiming-uniform at initialisation, so the model starts exactly at the pretrained function, and $\alpha = 2r$ (a constant scale $\alpha/r = 2$ across ranks). Only A, B are trained: $r \sum_{m \in \mathcal{T}} (d_{\text{in}}^m + d_{\text{out}}^m)$ per block, i.e. 1,792 parameters per unit of rank for 2 blocks (1,792 / 3,584 / 7,168 / 14,336 for $r = 1, 2, 4, 8$). Embeddings, layer norms and the head stay frozen, so the new task token keeps its random initial embedding. The target-module ablation restricts $\mathcal{T}$ to $\{q, v\}$ as in the original paper [3].

Last block. Only the second block, the final LayerNorm and the head are trained (34,496 parameters). Head only. Only the output projection (896 parameters). From scratch. A freshly initialised model, all parameters trained with the same data and budget; this controls for whether pretraining helps at all. No adaptation. The pretrained model evaluated as is.

Metrics. adapt_acc: exact match on the target task (2000 fresh test sequences); adapt_tok_acc: per-token accuracy; trainable_params; pretask_acc: exact match on sort ascending (token τ_0) after adaptation; forgetting = pretask accuracy before minus after adaptation.

IV. Experimental Setup

Data. Inputs are eight i.i.d. uniform digits. Pretraining uses 20,000 fixed sequences per seed (seed 1000 + s); adaptation uses a fixed set of $N = 2000$ sequences (seed 2000 + s); test sets of 2000 fresh sequences (seed 5000) are shared by all systems; a validation set (seed 7000) is used only for tuning. Train and test sequences are random and may overlap by chance (about 10^8 possible inputs).

Protocol. Seed $s \in \{0, \dots, 4\}$ fixes one pretrained model (4000 steps, batch 128, AdamW, lr 2×10^{-3} , cosine; 99.9% exact match on sort ascending), the adaptation set, and the adapter initialisation. Every adapted system, including from-scratch, uses 1500 steps, batch 64, AdamW with zero weight decay, 50 warm-up steps and cosine decay, and identical data; only the learning rate differs per system. Learning rates were chosen on the validation split of a separate seed (100, with its own pretrained model) from a grid of one run per value and task; after the best value fell at the grid edge for four systems the grid was extended once, but the selected rates for full fine-tuning (10^{-2}), LoRA (3×10^{-2}), last block (10^{-2}) and head (3×10^{-2}) are still at the upper edge of the extended grid, so they may be below the optimum (only scratch, 3×10^{-3} , is interior) (adding 10^{-2} for full and scratch and 3×10^{-2} for LoRA, last block and head; first grids: $\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$)

TABLE I

Main results, mean $\pm$ std over 5 seeds. Exact match on the target task (adapt_acc), per-token accuracy and exact match on the pretraining task after adaptation (pretask_acc, higher means less forgetting). Bold/underline: best/second best as computed by rh table, including the unadapted row; they do not imply a significant difference (LoRA $r = 4$, $r = 8$ and scratch tie at 0.997 on descending sort).

Method	Task	adapt_acc $\uparrow$	adapt_tok_acc $\uparrow$	pretask_acc $\uparrow$	n
Head only	sort_desc	0.012 $\pm$ 0.007	0.615 $\pm$ 0.015	0.000 $\pm$ 0.000	5
Last block	sort_desc	0.965 $\pm$ 0.007	0.995 $\pm$ 0.001	0.000 $\pm$ 0.000	5
From scratch	sort_desc	0.997 $\pm$ 0.001	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000	5
Full fine-tuning	sort_desc	0.996 $\pm$ 0.003	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000	5
LoRA $r=4$	sort_desc	0.997 $\pm$ 0.001	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000	5
LoRA $r=8$	sort_desc	0.997 $\pm$ 0.001	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000	5
LoRA $r=1$	sort_desc	0.978 $\pm$ 0.007	0.997 $\pm$ 0.001	0.000 $\pm$ 0.000	5
LoRA $r=2$	sort_desc	0.994 $\pm$ 0.002	0.999 $\pm$ 0.000	0.000 $\pm$ 0.000	5
No adaptation	sort_desc	0.000 $\pm$ 0.000	0.093 $\pm$ 0.015	0.999 $\pm$ 0.001	5
Head only	reverse	0.000 $\pm$ 0.000	0.248 $\pm$ 0.009	0.004 $\pm$ 0.004	5
Last block	reverse	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.001 $\pm$ 0.000	5
From scratch	reverse	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.001 $\pm$ 0.000	5
Full fine-tuning	reverse	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.001 $\pm$ 0.000	5
LoRA $r=4$	reverse	0.999 $\pm$ 0.002	1.000 $\pm$ 0.000	0.001 $\pm$ 0.000	5
LoRA $r=8$	reverse	0.941 $\pm$ 0.080	0.992 $\pm$ 0.011	0.001 $\pm$ 0.000	5
LoRA $r=1$	reverse	0.799 $\pm$ 0.447	0.821 $\pm$ 0.400	0.000 $\pm$ 0.000	5
LoRA $r=2$	reverse	0.800 $\pm$ 0.447	0.883 $\pm$ 0.261	0.000 $\pm$ 0.000	5
No adaptation	reverse	0.001 $\pm$ 0.000	0.193 $\pm$ 0.005	0.999 $\pm$ 0.001	5

for full and scratch, $\{10^{-3}, 3 \times 10^{-3}, 10^{-2}\}$ otherwise; see experiments/PROTOCOL.md). Selected rates: full 10^{-2} , scratch 3×10^{-3} , LoRA 3×10^{-2} (tuned at $r = 4$ only, reused for all ranks without per-rank tuning), last block 10^{-2} , head 3×10^{-2} . The tuning budget was 41 runs (about 10 minutes). Main experiments: 9 systems $\times$ 2 tasks $\times$ 5 seeds (90 runs); ablations 110 further runs; a typical run takes 6–27 s (5th–95th percentile; median 16 s), the six runs that include pretraining 50–60 s, and the 241 registered runs span about 54 minutes of wall clock (68 minutes of summed run time, above the 45 minutes planned in research.yaml) on 2 threads of a laptop CPU with up to two processes in parallel. Mean $\pm$ sample standard deviation over seeds are reported; paired tests are paired t -tests over seeds (as in rh compare), which with $n = 5$ have little power.

Ablations. (a) LoRA on $\{q, v\}$ only, ranks 1 and 4, five seeds. (b) Adaptation set size $N \in \{100, 300\}$ for six systems on descending sort, seeds 0–2, learning rates unchanged from the $N = 2000$ tuning. (c) Learning-rate sweep for full fine-tuning and LoRA $r = 4$, seeds 0–2. (d) Adaptation curves, evaluated every 50 steps on 500 test sequences, for seeds 0–2. Cells for $N = 2000$ and the default learning rates in these tables are the main-group runs restricted to seeds 0–2.

V. Results

Table I and Table II give the main results; Fig. 1 (which omits the from-scratch system) plots per-seed accuracy against trainable parameters. The pretrained model solves its own task (pretask accuracy 0.999) and none of the adapted tasks without adaptation (exact match 0.000 on descending sort, 0.001 on reversal).

H1 (LoRA $r = 4$ matches full fine-tuning with $< 15\%$ of parameters): supported in this setting for rank 4 (the registered statement says rank ≥ 4 ; rank 8 on reversal

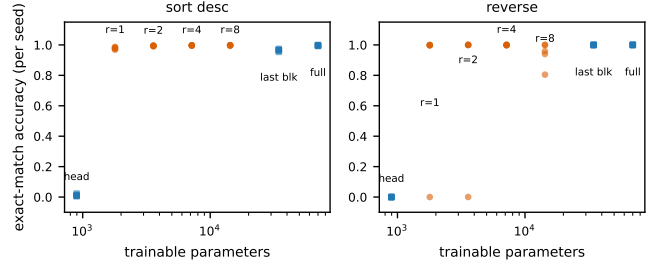


Fig. 1. Exact-match accuracy of every seed against trainable parameters (log scale). Orange: LoRA, blue: other systems. Labels give the rank or system.

(0.941, Table II) is below full fine-tuning (1.000) by more than the two-point margin, because of its seed failures, so H1 is not shown for rank 8 at the shared rate). LoRA $r = 4$ trains 7,168 parameters (10.2% of 70,016). Its exact match is 0.997 against 0.996 for full fine-tuning on descending sort and 0.999 against 1.000 on reversal (Table I); the paired differences are $+0.0003$ ($p = 0.816$) and -0.0007 ($p = 0.374$, Table III). Both are far inside the two-point margin, but five seeds cannot rule out small differences, and both systems are at the ceiling, so the task discriminates poorly between them. A model trained from scratch with the same budget also reaches 0.997 and 1.000, so at $N = 2000$ these tasks do not test whether pretraining is useful.

H2 (accuracy rises with rank): supported on descending sort, not interpretable on reversal. On descending sort the means are 0.978, 0.994, 0.997, 0.997 for $r = 1, 2, 4, 8$; rank 1 is below rank 8 by 0.019 (paired $p = 0.0037$). Ranks 4 and 8 are equal to the displayed precision. On reversal the means are 0.799, 0.800, 0.999, 0.941 (Table II). These are driven by whole-run failures, not by gradual loss: one seed at $r = 1$ and one at $r = 2$ end near chance with training loss far above the others (Table II, “fail”), and at $r = 8$ three seeds are at 0.80–0.96. The other seeds at $r = 1$ reach 0.997–1.000. Reversal is therefore solved by rank 1 whenever optimisation succeeds, and the apparent rank trend is an optimisation-stability effect at the learning rate selected on one validation seed; we did not isolate its cause, and it is not evidence that rank 1 lacks capacity. The LoRA rate was tuned at $r = 4$ only, so the failures at ranks 1, 2 and 8 may be artifacts of reusing it, and the rank trend on reversal should not be read as a property of rank. The rank-1 versus rank-8 difference on reversal is not significant ($p = 0.52$).

H3 (LoRA forgets less than full fine-tuning): not supported, and uninformative because of a floor effect. Every adapted system, LoRA at all ranks included, ends with pretask exact match of 0.000 on the descending-sort runs and at most 0.004 on average elsewhere (Table I); forgetting is 0.9995 for all of them on descending sort (identical across systems empirically, so no test is defined,

TABLE II

Parameters and accuracy by system, 5 seeds. “% full”: share of the 70,016 parameters. “min”: worst seed. “fail”: seeds with reversal accuracy below 0.5.

System	Params	% full	sort_desc	min	reverse	min	fail
Head only	896	1.3	0.012 ± 0.007	0.005	0.000 ± 0.000	0.000	5/5
Last block	34496	49.3	0.965 ± 0.007	0.955	1.000 ± 0.000	0.999	0/5
From scratch	70016	100.0	0.997 ± 0.001	0.994	1.000 ± 0.000	1.000	0/5
Full fine-tuning	70016	100.0	0.996 ± 0.003	0.993	1.000 ± 0.000	1.000	0/5
LoRA $r=1$	1792	2.6	0.978 ± 0.007	0.970	0.799 ± 0.447	0.000	1/5
LoRA $r=2$	3584	5.1	0.994 ± 0.002	0.992	0.800 ± 0.447	0.001	1/5
LoRA $r=4$	7168	10.2	0.997 ± 0.001	0.995	0.999 ± 0.002	0.997	0/5
LoRA $r=8$	14336	20.5	0.997 ± 0.001	0.995	0.941 ± 0.080	0.804	0/5

TABLE III

Paired t -tests over the five seeds (main group). Retention comparisons are identical across systems empirically (every system at the floor), so no test is defined.

A	B	metric	task	mean A-B	paired p
LoRA $r=4$	Full fine-tuning	adapt_acc	sort_desc	+0.0003	0.816
LoRA $r=4$	Full fine-tuning	adapt_acc	reverse	-0.0007	0.374
LoRA $r=4$	Last block	adapt_acc	sort_desc	+0.0314	0.000793
LoRA $r=4$	Last block	adapt_acc	reverse	-0.0005	0.546
LoRA $r=1$	LoRA $r=8$	adapt_acc	sort_desc	-0.0190	0.00369
LoRA $r=1$	LoRA $r=8$	adapt_acc	reverse	-0.1419	0.523
LoRA $r=4$	From scratch	adapt_acc	sort_desc	+0.0000	1
LoRA $r=4$	From scratch	adapt_acc	reverse	-0.0007	0.374
LoRA $r=4$	Full fine-tuning	pretask_acc	sort_desc	+0.0000	n/a
LoRA $r=4$	Full fine-tuning	pretask_acc	reverse	+0.0000	n/a

Table III). The residual 1/2000 of retention (0.0005) on reversal and in zero-shot rows is one test input whose reversal equals its ascending sort, not retained skill. Fig. 3 shows that retention is already zero at the first evaluation after 50 steps for full fine-tuning, LoRA $r = 1$ and $r = 4$, and last-block tuning. The task token did not protect the old skill, and neither the low rank nor freezing most weights helped; since even head-only tuning, which cannot adapt, sits at the floor and the pretrained model never learned to condition on the token, this test cannot discriminate between methods and we make no claim that LoRA does or does not forget less. The learning-rate sweep (Table VI) shows the same at every learning rate tried, including settings where adaptation itself was incomplete. We cannot say whether a shorter schedule, replay, or a larger model would change this.

H4 (last block worse than LoRA): supported on descending sort only. Last-block tuning trains 34,496 parameters (4.8 times LoRA $r = 4$) yet reaches 0.965 against 0.997 (difference +0.031 in favour of LoRA, paired $p = 0.0008$). On reversal the two are equal (0.999 vs. 1.000). Head-only tuning (896 parameters) fails on both tasks (0.012 and 0.000): with frozen features the output layer cannot realise a different permutation of the input.

Where the method fails. (i) LoRA at the selected learning rate has whole-run failures on reversal, at ranks 1, 2 and 8 but not 4 (Table II); with three seeds, $r = 4$ was stable at the same rate in the sweep (Table VI). (ii) With few examples, rank 8 collapses (Table V). (iii) Restricting LoRA to $\{q, v\}$ costs accuracy at low rank (Table IV).

TABLE IV

LoRA target modules (5 seeds), mean $\pm$ std exact match.

Rank	Targets	Params	sort_desc acc	reverse acc
1	all linear	1792	0.978 ± 0.007	0.799 ± 0.447
1	q, v only	512	0.577 ± 0.153	0.056 ± 0.091
4	all linear	7168	0.997 ± 0.001	0.999 ± 0.002
4	q, v only	2048	0.971 ± 0.005	1.000 ± 0.000

(iv) No system retained the pretraining task, so the design cannot separate methods on forgetting. (v) Learning rates for full fine-tuning, LoRA, last block and head are at the upper grid edge.

VI. Ablations and Analysis

Target modules. Table IV compares LoRA on all six linear maps with LoRA on $\{q, v\}$ only. At rank 1, $\{q, v\}$ (512 parameters) reaches 0.577 on descending sort and 0.056 on reversal, against 0.978 and 0.799 for all-linear (1,792 parameters). At rank 4, $\{q, v\}$ (2,048 parameters) reaches 0.971 and 1.000 against 0.997 and 0.999. The gap on descending sort at rank 4 is 0.026, with the two settings differing in parameters (2,048 against 7,168), so this ablation changes the parameter count together with the targets and does not separate the two. It does show that, at tiny rank, restricting LoRA to $\{q, v\}$ (a choice common in the LoRA literature) reaches lower accuracy than all-linear in this setting.

Adaptation set size. Table V (descending sort, seeds 0–2, same learning rates) examines low-data behaviour. With $N = 300$ the means are 0.765 (full fine-tuning), 0.511 (scratch), 0.838, 0.938 and 0.621 (LoRA $r = 1, 4, 8$); with $N = 100$ they are 0.204, 0.116, 0.385, 0.337 and 0.000. Pretrained full fine-tuning is ahead of scratch at both sizes, so pretraining is useful when data are scarce, in this three-seed sample. LoRA at ranks 1 and 4 has higher means than full fine-tuning at $N = 300$ and $N = 100$, but we ran no test that supports an ordering: standard deviations at $N = 100$ (0.15–0.18) are as large as the gaps, paired t -tests over the three seeds (computed from the registry rows, not reported in a table) give $p > 0.05$ for all four comparisons of LoRA $r = 1, 4$ with full fine-tuning, and the full fine-tuning rate (10^{-2}) was tuned for $N = 2000$

TABLE V

Descending-sort exact match against adaptation set size N (3 seeds; $N = 2000$ column from the main runs, seeds 0–2).

System	$N=100$	$N=300$	$N=2000$
Full fine-tuning	0.204 ± 0.027	0.765 ± 0.087	0.998 ± 0.001
From scratch	0.116 ± 0.010	0.511 ± 0.059	0.997 ± 0.000
Last block	0.075 ± 0.018	0.543 ± 0.069	0.965 ± 0.010
LoRA $r=1$	0.385 ± 0.184	0.838 ± 0.057	0.978 ± 0.009
LoRA $r=4$	0.337 ± 0.146	0.938 ± 0.007	0.996 ± 0.001
LoRA $r=8$	0.000 ± 0.000	0.621 ± 0.535	0.998 ± 0.000

TABLE VI

Learning-rate sweep, seeds 0–2 (the largest rate of each system is the main-group default). “Reverse retention”: sort-ascending exact match after adapting to reversal.

System	LR	sort_desc	reverse	reverse retention
Full fine-tuning	0.0003	0.956 ± 0.019	0.996 ± 0.002	0.001 ± 0.000
Full fine-tuning	0.001	0.987 ± 0.007	1.000 ± 0.000	0.001 ± 0.000
Full fine-tuning	0.003	0.993 ± 0.004	1.000 ± 0.000	0.001 ± 0.000
Full fine-tuning	0.01	0.998 ± 0.001	1.000 ± 0.000	0.001 ± 0.000
LoRA $r=4$	0.0003	0.517 ± 0.069	0.247 ± 0.272	0.001 ± 0.000
LoRA $r=4$	0.001	0.946 ± 0.024	0.999 ± 0.001	0.001 ± 0.000
LoRA $r=4$	0.003	0.980 ± 0.008	1.000 ± 0.000	0.001 ± 0.000
LoRA $r=4$	0.01	0.992 ± 0.002	1.000 ± 0.000	0.001 ± 0.000
LoRA $r=4$	0.03	0.996 ± 0.001	1.000 ± 0.000	0.001 ± 0.000

and may be too high for small N , so the comparison may favour LoRA unfairly. LoRA $r = 4$ at $N = 300$ (0.938 , std 0.007) has the largest gap in the means but is also not significant. Rank 8 is at or below 0.001 in all three seeds at $N = 100$ and in one of three seeds at $N = 300$ (std 0.535). In these failed runs the final training loss stayed high, so they look like optimisation failures at the shared high learning rate and not memorisation; we did not test whether a lower rate repairs them. Learning rates were not retuned per N .

Learning rate. Table VI and Fig. 2 sweep the learning rate for full fine-tuning and LoRA $r = 4$ (seeds 0–2). Full fine-tuning is stable over the range: 0.956 to 0.998 on descending sort, and at least 0.996 on reversal. LoRA is much more sensitive at the low end: at 3×10^{-4} it reaches 0.517 and 0.247 (std 0.272 on reversal), and on descending sort it reaches 0.992 at 10^{-2} and 0.996 at its default 3×10^{-2} , against 0.998 for full fine-tuning at its default. Pretask retention is about 0.001 at every learning rate in the table, for both methods, including where LoRA has barely adapted.

Adaptation speed. Fig. 3 (descending sort, seeds 0–2) shows that full fine-tuning reaches a mean target accuracy of 0.9 first (step 350), then last block (500) and LoRA at ranks 1 and 4 (550 each); the retention panel is zero for all systems at every evaluation point from step 50 on. Per-step differences are within the seed band shown, so we do not rank the three slower systems on speed.

VII. Limitations

The model has 70k parameters, tasks are two synthetic permutations of eight digits, and both are close to the ceiling at $N = 2000$, so the main comparison has little

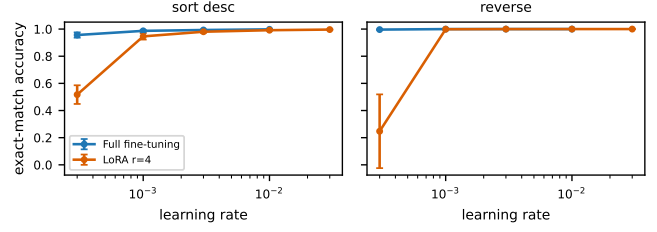


Fig. 2. Exact match against learning rate (mean $\pm$ std over 3 seeds).

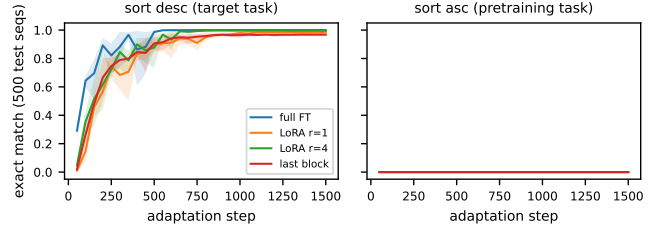


Fig. 3. Adaptation to descending sort (left) and retention of ascending sort (right) during training; mean with min-max band over seeds 0–2.

resolving power; the interesting differences appear only in subsampled or failing regimes, where learning rates were not retuned per N and the LoRA versus full fine-tuning ordering depends on that choice. Five seeds (three for the sweeps) give paired tests with little power, and the tests are not corrected for multiple comparisons. Every baseline is reimplemented here, with one learning rate per system chosen on one validation seed from a coarse grid; after one extension the selected rates for full fine-tuning, LoRA, last block and head are still at the upper grid edge with validation accuracy not yet saturated, so every system except scratch may be under-tuned, and the comparison is only as fair as this one-validation-seed tuning; the LoRA rate was tuned at rank 4 and reused for other ranks, which may explain the failures of ranks 1, 2 and 8 on reversal, and we did not test that explanation. Only the learning rate was tuned: the LoRA scale α/r , initialisation, number of steps and layer selection of the last-block baseline were fixed a priori. The pretrained model saw a single task token and was never trained to condition on it, and retention is at floor for every system including head-only tuning, so the forgetting comparison has no resolving power. Total forgetting may follow from our choices (1500 steps, no replay, fixed random embedding for the new task token, adapters active for every input) and says little about large models; our result does not contradict findings at scale [7]. We did not measure forgetting under early stopping or with a partial-learning setup, and no other PEFT variants (adapters, BitFit, DoRA) were run. A full study would add larger models and natural-language data, more seeds, per-method and per-rank learning-rate tuning, and mechanism

experiments (singular-value spectra of the full fine-tuning update, rank-wise learning-rate sweeps) to explain the failures.

VIII. Conclusion

In a 70k-parameter transformer, LoRA at rank 4 on all linear layers matched full fine-tuning on two adaptation tasks with 10.2% of the parameters, but a from-scratch model matched both at 2000 examples, so the tasks are easy at that data size. Accuracy rose with rank on descending sort (0.978 to 0.997), while on reversal whole-run optimisation failures at ranks 1, 2 and 8 hide any rank trend. Last-block tuning was worse than LoRA on descending sort and head-only tuning failed. Every adapted system lost the pretraining task almost completely, so the forgetting hypothesis is not supported and, because retention is at floor even for head-only tuning, the test is uninformative. In the low-data runs, LoRA rank 4 had the highest mean at 300 examples, but this is a non-significant three-seed observation at a full fine-tuning learning rate tuned for 2000 examples, and we draw no conclusion about LoRA versus full fine-tuning at small N . The code, run registry and tables reproduce all numbers.

References

- [1] N. Houlsby, A. Giurghi, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-Efficient Transfer Learning for NLP,” arXiv preprint arXiv:1902.00751, 2019. [Online]. Available: <https://arxiv.org/abs/1902.00751>
- [2] E. Ben-Zaken, S. Ravfogel, and Y. Goldberg, “BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models,” arXiv preprint arXiv:2106.10199, 2021. [Online]. Available: <https://arxiv.org/abs/2106.10199>
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” arXiv preprint arXiv:2106.09685, 2021. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [4] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick, and G. Neubig, “Towards a Unified View of Parameter-Efficient Transfer Learning,” arXiv preprint arXiv:2110.04366, 2021. [Online]. Available: <https://arxiv.org/abs/2110.04366>
- [5] A. Aghajanyan, L. Zettlemoyer, and S. Gupta, “Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning,” arXiv preprint arXiv:2012.13255, 2020. [Online]. Available: <https://arxiv.org/abs/2012.13255>
- [6] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen, “DoRA: Weight-Decomposed Low-Rank Adaptation,” arXiv preprint arXiv:2402.09353, 2024. [Online]. Available: <https://arxiv.org/abs/2402.09353>
- [7] D. Biderman, J. Portes, J. J. G. Ortiz, M. Paul, P. Greengard, C. Jennings, D. King, S. Havens, V. Chiley, J. Frankle, C. Blakeney, and J. P. Cunningham, “LoRA Learns Less and Forgets Less,” arXiv preprint arXiv:2405.09673, 2024. [Online]. Available: <https://arxiv.org/abs/2405.09673>
- [8] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, “Overcoming catastrophic forgetting in neural networks,” arXiv preprint arXiv:1612.00796, 2016. [Online]. Available: <https://arxiv.org/abs/1612.00796>
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” arXiv preprint arXiv:1706.03762, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [10] Z. Xu, H. Min, L. E. MacDonald, J. Luo, S. Tarmoun, E. Mallada, and R. Vidal, “Understanding the Learning Dynamics of LoRA: A Gradient Flow Perspective on Low-Rank Adaptation in Matrix Factorization,” arXiv preprint arXiv:2503.06982, 2025. [Online]. Available: <https://arxiv.org/abs/2503.06982>
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” arXiv preprint arXiv:1711.05101, 2017. [Online]. Available: <https://arxiv.org/abs/1711.05101>